

NumPy Theory Questions and Answers (1-25)

1. What is NumPy and why is it used in Python for data analysis?

NumPy (Numerical Python) is a powerful library for numerical computations. It provides support for multidimensional arrays and matrices, along with a large collection of mathematical functions to operate on them. It is fast, memory-efficient, and forms the backbone of libraries like pandas, scikit-learn, and TensorFlow.

2. What is the difference between a Python list and a NumPy array? Why would you prefer one over the other?

Python lists can store multiple data types and are dynamically sized, but are slower. NumPy arrays store homogeneous data and are optimized for vectorized operations and memory efficiency. Prefer NumPy when dealing with large numeric datasets or matrix operations, due to performance and functionality.

3. How does NumPy achieve better performance than built-in Python structures?

NumPy arrays are implemented in C and use contiguous memory allocation, allowing operations to be vectorized and eliminating the overhead of Python loops. This leads to significant performance gains over lists.

4. Explain the concept of vectorization in NumPy. How does it differ from traditional looping?

Vectorization allows you to perform operations on entire arrays without explicit loops, using optimized C implementations under the hood. Example: `arr + 5` adds 5 to each element (vectorized). This is faster and more concise than using a for-loop.

5. What are the different ways to create NumPy arrays? Name at least 5 methods.

`np.array()`, `np.zeros()`, `np.ones()`, `np.arange()`, `np.linspace()`, `np.random.rand()`, `np.eye()`

6. What does the `np.arange()` function do? How is it different from `np.linspace()`?

`np.arange(start, stop, step)` creates values with a specific step. `np.linspace(start, stop, num)` generates evenly spaced values between two points. Use `arange()` when controlling steps, and

`linspace()` when you want a fixed number of elements.

7. What are the `shape` and `dtype` attributes of a NumPy array? Explain with examples.

`shape` returns a tuple representing the array's dimensions; `dtype` returns the data type of the array elements. Example: `arr = np.array([[1, 2], [3, 4]])`; `arr.shape` -> (2, 2); `arr.dtype` -> int64

8. What are broadcasting rules in NumPy? How does broadcasting work between arrays of different shapes?

Broadcasting allows operations on arrays of different shapes by virtually replicating data. The shapes are compared right to left, and if they match or one is 1, broadcasting is possible.

9. What is the difference between `reshape()`, `ravel()`, and `flatten()`?

`reshape()` changes shape without modifying data. `flatten()` returns a copy of the array, always 1D. `ravel()` returns a view (if possible), more memory-efficient.

10. How do you handle missing or NaN values in NumPy arrays?

Use `np.isnan()` to detect NaNs; `np.nanmean()`, `np.nanstd()` to calculate ignoring NaNs; `arr[np.isnan(arr)] = value` to replace NaNs.

11. What is the difference between `np.sum()` and `np.cumsum()`? Give examples.

`np.sum()` gives the total sum. `np.cumsum()` gives the cumulative sum at each index. Example: `np.sum([1, 2, 3])` -> 6; `np.cumsum([1, 2, 3])` -> [1, 3, 6]

12. How can you perform element-wise operations between two NumPy arrays?

By directly using operators: `a + b`, `a - b`, `a * b`, `a / b`. These operations are broadcasted and work as long as array shapes are compatible.

13. How do you apply conditional logic (e.g., filtering or replacing values) in NumPy?

Using boolean indexing or `np.where()`: `arr[arr > 5]`; `np.where(arr > 5, 1, 0)`

14. What is the purpose of `np.where()` and how is it used?

`np.where(condition, x, y)` returns elements from `x` or `y` depending on the condition. It's a vectorized ternary operator.

15. How can you generate identity matrices, diagonal matrices, or random matrices using NumPy?

Identity: `np.eye(n)`; Diagonal: `np.diag([1,2,3])`; Random: `np.random.rand(m, n)` or `np.random.randint()`

16. What are the differences between `np.copy()` and just using `=` assignment in NumPy arrays?

`=` creates a reference. `np.copy()` creates a new independent object.

17. What are structured arrays in NumPy? When would you use them?

Structured arrays allow storage of heterogeneous data in a single array using custom dtypes. Useful for tabular data.

18. What are views and copies in NumPy? How can modifying one affect the other?

A view shares the same data buffer; changing one affects the other. A copy is a new array with its own data.

19. How do you find unique elements, frequencies, and intersections between arrays using NumPy?

`np.unique(arr)` for unique values; `np.unique(arr, return_counts=True)` for frequencies; `np.intersect1d(arr1, arr2)` for intersections.

20. What is broadcasting in NumPy and what are its limitations or potential pitfalls?

Broadcasting allows arrays of different shapes to be operated together. Misaligned shapes can cause errors. Use `reshape()` or `expand_dims()` to resolve.

21. How does NumPy handle memory management and why is it more efficient for large datasets compared to lists or pandas Series?

NumPy uses contiguous memory blocks and fixed data types, allowing efficient access and computation. Python lists store references to objects, causing memory overhead.

22. Explain the concept of boolean indexing in NumPy. How is it different from integer

indexing?

Boolean indexing filters data using boolean conditions. Integer indexing selects elements using specific positions.

23. What is the difference between `.T`, `transpose()`, and `swapaxes()` in NumPy? When would you use each?

`.T` and `transpose()` are for general transposition; `swapaxes()` is used for n-D arrays to swap specific axes.

24. How can NumPy be used to efficiently calculate the pairwise distance between rows of a matrix?

Using broadcasting or `scipy.spatial.distance.cdist`. Broadcasting avoids loops and is memory efficient.

25. What are broadcasting limitations and how can you resolve shape mismatch errors in NumPy?

Shapes must be compatible from the right. Use `reshape()`, `np.newaxis`, or `tile()` to align dimensions.